Reviving STARMAP with Claude

Oct 12, 2026 · @Jack

In two working days, Claude Code rewrote STARMAP, a 1978 APL program, as modern Dyalog APL 21 that reproduces the original's output exactly, byte for byte, under human review at every stage.

Starting point

The original was the 1978 STARMAP listings, a classic example of what Paul Berry called expository programming: it plots the night sky for a given place and time. Stephen Taylor transcribed the book; Neil Kirsopp made the code run in Dyalog APL, fixed transcription errors, and added an SVG map, an animation and a web app. That version is kept unchanged as the reference.

Its 95 functions are 1970s style: `→` branches in 10 of them, data passed through more than a dozen globals, orbital selectors that read `ORB` from their caller by dynamic scope, and two functions of 226 and 642 lines. Twelve tests compare its results with JPL DE421 positions for the stars, Sun, Moon and planets.

How the work ran

The work ran in six stages, each with its own GitHub issue and branch. Claude wrote the tests first and stopped; Stephen reviewed them; only then did Claude write the code. Before each review, Claude ran its new tests against a stand-in built from the book's own functions, which caught bugs in the tests themselves. Most tests run book and modern code on the same inputs and require identical results (`≡`).

1. A test runner that loads both versions side by side in script mode
2. Core calculations: orbital selectors take the element table as an argument; loops and branches replaced
3. One function, `SkyAt`, returns the whole sky as a namespace, replacing three functions and their globals
4. Terminal reports become functions that return lines, compared character for character with the book's printed output
5. Every function becomes a dfn with a bicameral name; constants become upper-case variables
6. SVG maps, animation and web: the animated SVG, the frame files, and the PNG images and MP4 video made from them are identical to the book version's, byte for byte

Before and after

The modern version is a third shorter, its longest function is an eighth the length, and no data passes through globals.

	Book version (`APLSource/`)	Modern version (`Starmap/`)
Functions	95 tradfns	92 dfns, 10 niladic tradfns
Source lines	1,848	1,184
Longest function	642 lines	81 lines
Functions using `→`	10	0
Data flow	globals and dynamic scope	arguments and results
Names	`KEPLINVERSE` , `RADECDIST`	`KeplInverse` , `RaDecDist`
Tests	12 (against JPL)	12 more (against the book version and JPL)

The 300 lines of JavaScript inside the animated SVG are now Dyalog 20 array notation, one string per line. The 10 tradfns take no argument: input prompts, entry points, the web server, and `Initialise` , which sets the constants.

Who did what

Stephen made the decisions and Claude did the reading, writing and checking; the decisions shaped the result more than any line of code.

Stephen decided:

- **Replicate first.** The modern code must reproduce the book's results exactly, bugs included; each bug becomes an issue to fix later.
- **Modernise boldly.** Halfway through, he reversed his own rulings: all the code was in scope, and every tradfn should become a dfn, not just be tidied.
- **Constants.** `PI` and `EARTH` became upper-case variables set by one function, `Initialise` , which Link loads like any other function.
- **Every test change.** Claude could change a test only with his written approval. He reviewed every set of tests, and opened and merged every pull request himself.

Claude found:

- **Three bugs in the book version**, now reproduced on purpose and logged as issues: a Newton iteration that stops when any element converges (#19), a footer that shows the observation's weekday rather than the print date's (#22), and an animation option that is parsed but never used (#26).
- **Two Dyalog 21 surprises inside dfns.** An embedded assignment after a function call (`x←F t←⍵`) and a strand of indexed names (`a[;1]b[;2]`) behave differently in a dfn than in a tradfn; such lines are split.
- **Its own mistakes.** The stand-in runs caught errors in its tests, and a review before pushing caught `Entry` asking for longitude first: `GetDate,GetTime,GetLat,GetLong` runs right to left.